KERALA POLYTECHNIC · REVISION 2026

Embedded System and Real time Operating System

Complete student lesson for course code **5131**.

Revision 2026 Semester 5 Program Core 4 modules

Course snapshot

Scheme: 4 (L4:T:0:P:0)

Credits: 4

Contact: 60 periods per semester

Module hours listed: 58

Official Source Declaration

This lesson uses the supplied official syllabus PDF as the authoritative source for course information, revision, modules, topics, objectives, Course Outcomes, assessment information, official activities, practical requirements, and references.

Source handling: Official wording and numerical values are retained wherever supplied. Educational explanations, Malayalam support, examples, diagrams, and revision questions are added as study support and are not presented as additional official requirements.

Transparency note: Official assessment information is reproduced below from the supplied syllabus. Any limitation in the supplied PDF is not silently corrected.

How to Study This Lesson

Recommended sequence

1. Begin with the official source declaration and course dashboard.
2. Study each module in the official order and keep the coverage table as a checklist.
3. Open every topic, understand the example or procedure, and write your own short answer.
4. Use the revision section, glossary, questions, and practice paper after completing the modules.

Study principle: Keep English technical terminology, symbols, units, and official assessment wording unchanged in examination answers. Use the Malayalam support blocks only as a bridge to understanding.

Course Dashboard

Course code

5131

Semester

5

Credits

4

Teaching scheme

4 (L4:T:0:P:0)

Module-hour and outcome mapping

No.	Module	Official hours	Relevant CO / level
1	Embedded-System Concepts, Building Blocks and AVR Architecture	12	CO1
2	AVR Programming in C, Timers and Interrupts	19	CO2
3	Microcontroller Interfacing with External Peripherals	12	CO3
4	Real-Time Operating System Concepts	15	CO4

Prerequisites

- Digital Electronics — Digital Computer Principles, semester III.
- Programming Concepts — Programming in C, semester III.

How to study this lesson

1. Read the module introduction and learning goals.
2. Open every topic and reproduce its example, sequence, or procedure.
3. Use Malayalam support as a bridge; retain English technical terms for examinations.
4. Attempt the module questions without looking at the answers.

Objectives, Course Outcomes and CO–PO Information

Official course objectives

1. Introduce the technologies behind embedded computing systems.
2. Provide knowledge on the working of microcontrollers and its applications.
3. Familiarize the key concepts of embedded systems such as I/O, timers, interrupts and interaction with peripheral devices.
4. Introduce the basic concepts of Embedded Operating Systems.

Course Outcomes

1. CO1: Explain the basic concepts and structure of Embedded systems — 12 hours — Understanding.
2. CO2: Develop Programs for AVR Microcontrollers in C — 19 hours — Applying.
3. CO3: Illustrate the interfacing of Microcontroller with external peripherals — 12 hours — Applying.
4. CO4: Explain the key concepts of Real Time Operating Systems — 15 hours — Understanding.

CO–PO mapping as supplied

CO1: PO1 = 2 (Moderately mapped).
CO2: PO1 = 3 (Strongly mapped).
CO3: PO1 = 3 (Strongly mapped).
CO4: PO1 = 2 (Moderately mapped).
Legend: 3-Strongly mapped, 2-Moderately mapped, 1-Weakly mapped.

Complete Syllabus Coverage

Use this checklist to confirm that every official module topic is represented in the lesson.

Complete official topic coverage checklist

Module	Topic code	Topic	Hours
module-1	M1.01	Features of embedded systems	3
module-1	M1.02	Building blocks of embedded systems	4
module-1	M1.03	AVR microcontroller architecture	5
module-2	M2.01	C data types for the AVR microcontroller	1
module-2	M2.02	C programs for time delay and I/O operations	4
module-2	M2.03	C programs for logic and arithmetic operations	2
module-2	M2.04	Data conversion and data serialisation	2
module-2	M2.05	Normal and CTC modes of timers	2
module-2	M2.06	C programs for time delays and event counting using Timer/Counter	4
module-2	M2.07	Interrupts in AVR	1
module-2	M2.08	Interrupt programming in C	3
module-3	M3.01	RS232, serial port, LCD and keyboard interfacing	6
module-3	M3.02	ADC, DAC and sensor interfacing	6
module-4	M4.01	Functions of an Operating System	1
module-4	M4.02	Types and features of Operating Systems	1
module-4	M4.03	Tasks, processes and threads	3
module-4	M4.04	Multiprocessing and multitasking	1

Module	Topic code	Topic	Hours
module-4	M4.05	Task-scheduling algorithms	3
module-4	M4.06	Task communication and synchronization	4
module-4	M4.07	Device drivers	1
module-4	M4.08	Functional and nonfunctional requirements when selecting an RTOS	1

Learning Roadmap

1. Embedded-System Concepts, Building Blocks and AVR Architecture
CO1 · 12 hours

2. AVR Programming in C, Timers and Interrupts
CO2 · 19 hours

3. Microcontroller Interfacing with External Peripherals
CO3 · 12 hours

4. Real-Time Operating System Concepts
CO4 · 15 hours

The roadmap follows the order of the supplied syllabus.

OFFICIAL MODULE 1

Embedded-System Concepts, Building Blocks and AVR Architecture

An embedded system combines computation with a product or process to perform a defined function under resource, timing, power, reliability, and interface constraints. The module then connects system building blocks to the AVR architecture.

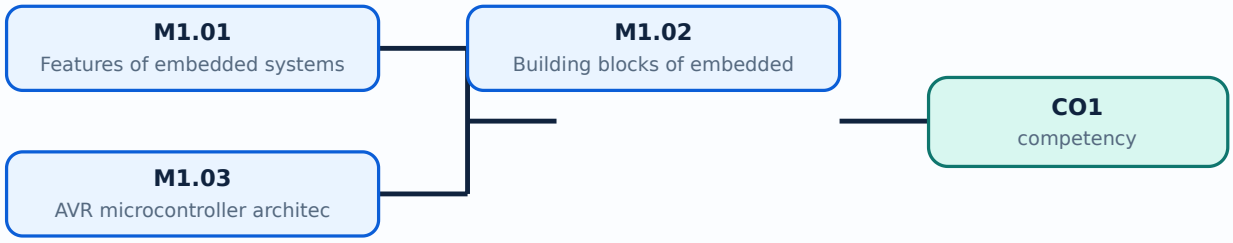
Hours: 12
CO1 · Bloom level: Understanding

Learning goals

- Define and classify embedded systems and state applications.
- Identify core, memory, sensors, actuators, I/O, communication, firmware, and quality attributes.
- Interpret the main registers and memory areas of the AVR/ATmega32 architecture.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-1: the listed topics build the module competency.

M1.01 — Features of embedded systems (3 hours)

Concept and explanation

An embedded system is a computer-based system designed to perform a dedicated function within a larger product. Compared with a general-purpose computer, it commonly has a defined application, constrained memory and power, real-time or predictable response needs, specialised I/O, and long service life. Systems can be classified by application, complexity, performance, connectivity, timing, and scale. Applications include appliances, vehicles, industrial controllers, medical devices, consumer products, and communication equipment.

Malayalam support: Embedded system □□□ larger product-□□□□ □□□□□□ dedicated function □□□□□□□□□ computer-based system □□□. Memory, power, timing, I/O, reliability constraints □□□□□□□□□□ □□□□□□□□.

Study points

- Compare embedded and general-purpose computers; classify by application, complexity, and timing.

Engineering / workplace application: Industrial control, automotive electronics, appliances, and medical instruments depend on embedded controllers.

Illustrative example: A washing-machine controller reads sensors, executes a control sequence, drives actuators, and displays status without being a general-purpose desktop.

Common mistake: An embedded system is not defined only by having a microcontroller; the application, constraints, interfaces, and firmware are also important.

M1.02 — Building blocks of embedded systems (4 hours)

Concept and explanation

Building blocks include the processing core, memory, sensors, actuators, I/O subsystem, communication interface, embedded firmware, power supply, clock and reset, and protection circuits. Memory may be classified as program memory, data memory, volatile, non-volatile, internal, or external. Memory selection depends on capacity, speed, endurance, power, cost, and retention. Sensors measure physical variables; actuators produce physical action; interfaces connect the controller to the environment and other systems.

Malayalam support: Core, memory, sensors, actuators, I/O, communication, firmware, power, clock/reset □□□□□□ embedded system blocks □□□. Memory □□□□□□□□□□□□□□□□□□□□ capacity, speed, endurance, power, cost, retention □□□□□□□.

Study points

- Draw a block diagram and show data/control flow.
- Explain sensor-to-controller-to-actuator interaction.
- Relate memory type to program, data, and retention requirements.

Engineering / workplace application: A motor controller uses sensors as inputs, firmware for decisions, power electronics as actuators, and communication for diagnostics.

Illustrative example: A temperature sensor feeds an ADC, firmware compares the value with a limit, and an actuator turns a cooling fan on or off.

Common mistake: Listing components without explaining their function or signal direction gives an incomplete system description.

M1.03 — AVR microcontroller architecture (5 hours)

Concept and explanation

The AVR architecture is examined through selection factors, a simplified microcontroller view, and ATmega32 resources. Important elements include registers, data memory, status register, program counter, program ROM space, I/O ports, and registers associated with I/O ports. The program counter points to the next instruction; status flags record arithmetic or logical results; I/O direction and data registers configure and access pins. Architecture knowledge allows a C program to be related to hardware behaviour.

Malayalam support: AVR architecture □□□□□□□□□□□□ program counter, status register, program memory, data memory, general registers, I/O direction/data registers □□□□□□□□□□ role □□□□□□□□□□□□□□. C code hardware registers-□□□□□ □□□□□□□□□□□□□□□.

Study points

- Identify program counter, status register, program memory, data memory, and I/O registers.
- Explain why architecture affects C programming.
- Trace a simple input-read and output-write operation.

Engineering / workplace application: AVR microcontrollers are used in educational control systems, instrumentation, and compact embedded products.

Illustrative example: A C statement that sets an output bit ultimately changes an I/O register and the corresponding microcontroller pin state.

Common mistake: Confusing program memory with data memory causes incorrect explanations of where code and variables reside.

Module summary: Embedded systems are understood by connecting application constraints to hardware blocks, firmware, memory, I/O, and architecture.

OFFICIAL MODULE 2

AVR Programming in C, Timers and Interrupts

This module turns architecture into C programs for I/O, arithmetic, data movement, timers, counters, and interrupt service routines.

Hours: 19

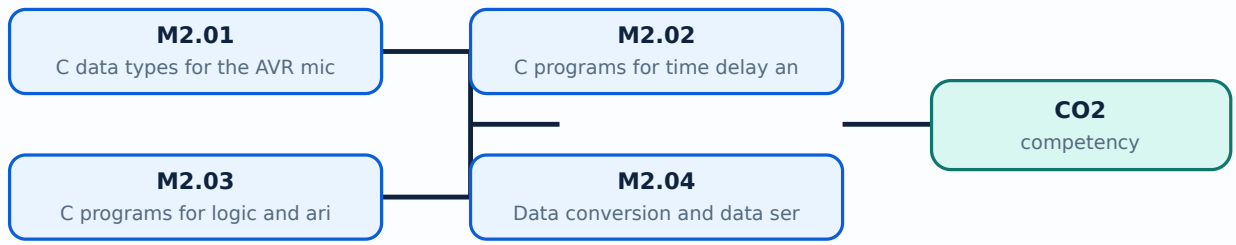
CO2 · Bloom level: Applying

Learning goals

- Use AVR C data types and write time-delay and I/O programs.
- Implement logic, arithmetic, conversion, serialisation, timer, and counter operations.
- Explain interrupts, ISR flow, priority, and interrupt programming in C.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-2: the listed topics build the module competency.

M2.01 — C data types for the AVR microcontroller (1 hours)

Concept and explanation

AVR C data types should be selected according to range, signedness, memory use, and required operation. Integer widths, character data, Boolean conditions, and fixed-width types help make embedded code predictable. The programmer must consider overflow, truncation, volatile hardware registers, and the difference between a value and a memory-mapped register.

Malayalam support: AVR C-□□□ data type □□□□□□□□□□□□□□□□□□ range, signedness, memory use, overflow, truncation □□□□□□ □□□□□□□□□□□□. Hardware register access-□ volatile concept □□□□□□□□□□.

Study points

- Choose a type for a given range and signedness.
- Explain overflow and truncation with a small example.
- Distinguish ordinary variables from hardware registers.

Engineering / workplace application: Correct types reduce RAM use and prevent unexpected arithmetic in small microcontrollers.

Illustrative example: An 8-bit timer counter wraps after its maximum value, so the program must handle rollover when measuring elapsed time.

Common mistake: Using a signed type for an unsigned counter or ignoring overflow can produce incorrect control behaviour.

M2.02 — C programs for time delay and I/O operations (4 hours)

Concept and explanation

I/O programming configures a port direction, writes output data, reads input data, and applies timing between changes. A delay can be created by calibrated loops or, more reliably, by a timer. Embedded code must account for clock frequency, instruction cycles, switch bounce, and hardware active-high or active-low logic. A good program uses named masks and avoids changing unrelated bits in a shared port register.

Malayalam support: I/O program □□□□□ port direction set □□□□□□□, input read/output write □□□□□□□. Delay clock frequency-□□□□□□ □□□□□□□□□□□□ □□□□□□□□; switch bounce and active-high/low logic □□□□□□□□□□□□.

Study points

- Explain input and output configuration.
- Use bit masks to change only required port bits.
- State why timer-based delay is more predictable than long software loops.

Engineering / workplace application: LEDs, switches, relays, and status indicators are common introductory I/O devices.

Illustrative example: A masked operation can set one LED bit while retaining the state of other port pins used by a display.

Common mistake: Writing an entire port value can unintentionally change other output pins.

M2.03 — C programs for logic and arithmetic operations (2 hours)

Concept and explanation

Embedded programs use arithmetic and logical operators to transform sensor values, generate control decisions, and manipulate bits. Bitwise AND masks inputs, OR sets selected bits, XOR toggles bits, and shifts move fields. Arithmetic should be checked for overflow, signedness, and scaling. Conditions should be written so that the hardware-safe state is explicit when a sensor value is invalid or out of range.

Malayalam support: Bitwise AND mask □□□□□□□□□ OR set □□□□□□□□□ XOR toggle □□□□□□□□□ shift field □□□□□□□□□ □□□□□□□□□□□□□□□. Arithmetic-□ overflow, signedness, scaling □□□□□□□□□□□□.

Study points

- Differentiate logical and bitwise operations.
- Use masks for selected bits.
- Include range and overflow checks in control code.

Engineering / workplace application: Bit operations are used in register configuration, protocol fields, sensor flags, and compact status words.

Illustrative example: To test whether bit 3 is set, mask the value with `0x08` and compare the result with zero.

Common mistake: Using logical `&&` where bitwise `&` is required changes the result and may corrupt register configuration.

M2.04 — Data conversion and data serialisation (2 hours)

Concept and explanation

Data conversion changes representation, such as ADC counts to voltage, integer to character digits, binary fields to hexadecimal, or sensor units to engineering units. Serialisation packs structured data into a byte sequence for transmission or storage; deserialisation reconstructs it. The design must define byte order, field width, sign, scaling, checksum or error detection, and framing.

Malayalam support: Data conversion representation □□□□□□□□□□; serialisation structured data bytes □□□ □□□□□□□□□□. Byte order, field width, sign, scaling, framing, checksum □□□□□□ define □□□□□□□.

Study points

- Give an example of sensor-count-to-unit conversion.
- Explain field order and framing in serialised data.
- Validate range and communication errors.

Engineering / workplace application: Embedded controllers serialise measurements for displays, logging, diagnostics, and controller networks.

Illustrative example: A 16-bit sensor value can be sent as high byte followed by low byte with a defined checksum and a receiver that reconstructs the same value.

Common mistake: Sending raw bytes without defining byte order or scaling makes the receiver interpret data incorrectly.

M2.05 — Normal and CTC modes of timers (2 hours)

Concept and explanation

In Normal timer mode, the counter increments through its range and overflows, which can trigger a flag or interrupt. In Clear Timer on Compare (CTC) mode, the counter resets when it matches a programmed compare value, producing a repeatable timing interval. Timer configuration includes clock source, prescaler, counter register, compare register, waveform mode, and interrupt enable. The interval depends on clock frequency, prescaler, and count value.

Malayalam support: Normal mode overflow □□□ count □□□□□□□; CTC mode compare match □□□□□□□ clear □□□□□□□. Clock, prescaler, counter, compare value, waveform mode, interrupt enable □□□□□□ configure □□□□□□□.

Study points

- Compare overflow and compare-match operation.
- Identify timer registers and prescaler role.
- Relate timer interval to clock and count settings.

Engineering / workplace application: Timers generate periodic sampling, PWM-related timing, communication timing, and scheduler ticks.

Illustrative example: A CTC compare value can be selected so that a timer interrupt occurs at a fixed periodic rate for sensor sampling.

Common mistake: Changing a prescaler without recalculating the interval causes incorrect timing.

— M2.06 — C programs for time delays and event counting using Timer/Counter (4 hours)

Concept and explanation

A timer can generate a delay by counting clock ticks; a counter can count external events. A C program configures the mode and registers, starts the timer, waits for a flag or services an interrupt, then clears or records the event. Time calculations must use the microcontroller clock and prescaler. Event counters require debouncing or signal conditioning when the input is mechanical or noisy.

Malayalam support: Timer internal clock ticks count □□□□□□ delay □□□□□; counter external events count □□□□□□□. Clock, prescaler, overflow, flag clearing, input noise □□□□□□ □□□□□□□□□□□.

Study points

- Write the configuration sequence conceptually.
- Calculate a count from clock and desired interval.
- Explain why noisy external pulses require conditioning.

Engineering / workplace application: Timers and counters measure frequency, generate periodic tasks, and schedule control operations.

Illustrative example: A counter can record encoder pulses while a timer periodically reads and reports the count to estimate speed.

Common mistake: Polling a flag without clearing or servicing it correctly can make the program repeat an old event.

M2.07 — Interrupts in AVR (1 hours)

Concept and explanation

An interrupt temporarily transfers control from the main program to an Interrupt Service Routine (ISR) when an enabled event occurs. AVR interrupt sources can include external pins, timers, serial communication, ADC completion, and other peripherals. The controller saves the required context, identifies the vector, executes the ISR, and returns to the interrupted program. Interrupt priority and shared-data protection must be understood.

Malayalam support: Interrupt event □□□□□□□□ main program-□ □□□□□□ ISR-□□□□□□ control □□□□□. Timer, external pin, serial, ADC □□□□□□□□ sources □□□□□□□□. ISR □□□□□□□ □□□□□□□□□□□□□□□□□□□□□.

Study points

- List interrupt sources and the basic execution sequence.
- Explain enable, vector, ISR, and return.
- Mention shared-data and priority concerns.

Engineering / workplace application: Interrupts allow the CPU to respond to events without continuously polling every device.

Illustrative example: A timer interrupt can set a flag for the main loop instead of performing a complete communication transaction inside the ISR.

Common mistake: Doing long blocking work inside an ISR can delay other important events.

M2.08 — Interrupt programming in C (3 hours)

Concept and explanation

Interrupt programming enables the source, defines the vector or handler, writes a short ISR, clears the event condition, and safely communicates results to the main program. Shared variables may need volatile qualification and atomic access or temporary interrupt masking. The ISR should be deterministic and should avoid unsafe library calls or long loops. Testing must include simultaneous or rapid events and missed-event conditions.

Malayalam support: Interrupt C programming-□ source enable, vector/handler, short ISR, flag clear, main loop communication □□□□□□ □□□□. Shared variable-□□□□ volatile and atomic access □□□□□□□□□□□□.

Study points

- Outline the ISR programming steps.
- Explain why ISR length and shared data matter.
- Test rapid, simultaneous, and missed events.

Engineering / workplace application: Embedded systems use interrupt-driven serial reception, timer ticks, keyboard input, and emergency signals.

Illustrative example: An ISR can increment a counter and set a flag; the main loop later formats the count for display or communication.

Common mistake: Forgetting to clear the interrupt source can immediately re-enter the ISR.

Module summary: AVR C programming becomes reliable when data types, bit operations, timing, registers, flags, and ISR boundaries are explicitly controlled.

OFFICIAL MODULE 3

Microcontroller Interfacing with External Peripherals

Interfacing connects the microcontroller to serial links, displays, keyboards, converters, and sensors. Correct electrical levels, timing, protocol, and software

configuration are as important as the C code.

Hours: 12

CO3 · Bloom level: Applying

Learning goals

- Interface RS232/serial port, LCD, and keyboard.
- Interface ADC, DAC, and sensors.
- Write or explain the associated C-programming sequence.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.

Learning map for Module module-3: the listed topics build the module competency.

M3.01 — RS232, serial port, LCD and keyboard interfacing (6 hours)

Concept and explanation

RS232 interfacing requires suitable voltage-level conversion because microcontroller logic levels and RS232 signal levels differ. Serial programming configures baud rate, frame format, transmit/receive control, and status flags. LCD interfacing uses command and data sequences, enable timing, and data/control lines. Keyboard interfacing scans rows and columns or reads key signals and must handle debounce.

Malayalam support: RS232 voltage level microcontroller logic-□ □□□□□□ □□□□□□□□□□□□□□□□□ level conversion □□□□. Serial-□ baud/frame/status; LCD-□ command/data/timing; keyboard-□ scan and debounce □□□□□□□□□□.

Study points

- Explain serial configuration and level conversion.
- Differentiate LCD command and data transfers.
- Describe keyboard scanning and debouncing.

Engineering / workplace application: Serial ports support diagnostics and device communication; displays and keyboards provide human-machine interaction.

Illustrative example: A serial receiver checks a status flag, reads the data register, and stores the byte while the main program updates an LCD message.

Common mistake: Connecting RS232 directly to logic pins without a level converter can damage hardware or produce invalid signals.

M3.02 — ADC, DAC and sensor interfacing (6 hours)

Concept and explanation

An ADC converts an analogue sensor voltage into a digital code for the microcontroller. Resolution, reference voltage, sampling time, input range, quantisation, and noise affect the result. A DAC converts a digital value to an analogue output for a control or signal-generation stage. Sensor interfacing also requires calibration, filtering, signal conditioning, correct grounding, and safe voltage levels. C code selects a channel, starts conversion, waits for completion, reads the result, scales it, and applies a control decision.

Malayalam support: ADC analogue voltage digital code □□□□□□; DAC digital value analogue output □□□□□□. Resolution, reference, sampling, noise, calibration, filtering, grounding □□□□□□ □□□□□□□□□□□□.

Study points

- Trace ADC conversion and scaling steps.
- Explain DAC use and signal conditioning.
- Relate sensor range to reference and resolution.

Engineering / workplace application: Temperature, light, pressure, and position sensors are interfaced through ADC channels in instrumentation and control systems.

Illustrative example: If a sensor output spans 0 to 5 V and the ADC reference is 5 V, the digital reading can be scaled to estimate the measured voltage before calibration.

Common mistake: A high-resolution ADC cannot correct a badly calibrated sensor or an input voltage outside its safe range.

Module summary: Peripheral interfacing is a complete path from electrical signal and protocol to register configuration, timing, C code, and validated output.

OFFICIAL MODULE 4

Real-Time Operating System Concepts

An RTOS supports predictable execution of multiple tasks and communication with hardware. The central focus is timing, scheduling, synchronisation, drivers, and selecting an RTOS for functional and nonfunctional requirements.

Hours: 15

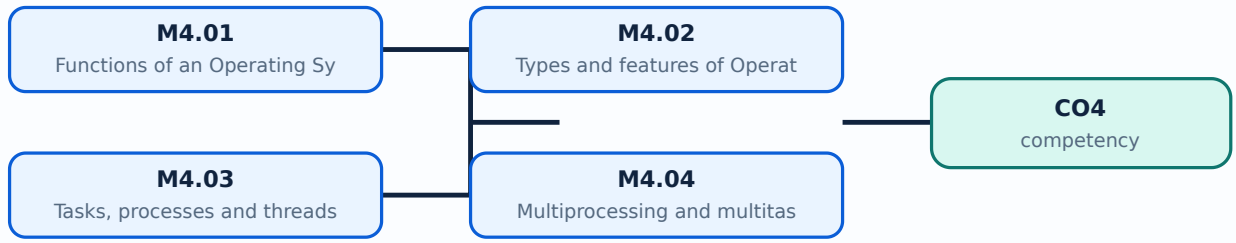
CO4 · Bloom level: Understanding

Learning goals

- Explain OS functions and types.
- Differentiate task, process, thread, multiprocessing, and multitasking.
- Describe scheduling, communication, synchronisation, device drivers, and RTOS-selection criteria.

Module study tip

Read every topic in sequence, reproduce its example or procedure, and write a short explanation before attempting the module questions.



Learning map for Module module-4: the listed topics build the module competency.

M4.01 — Functions of an Operating System (1 hours)

Concept and explanation

An operating system manages processor time, memory, files or resources, input/output devices, protection, user or task interfaces, and error handling. In an embedded or real-time system, the OS also provides task management, timing services, inter-task communication, synchronisation, and device access. The OS creates an abstraction so applications can use resources through defined interfaces rather than controlling every hardware detail directly.

Malayalam support: Operating system processor, memory, I/O, protection, resource, error management □□□□□□□□□□. RTOS-□ task, timing, communication, synchronisation services □□□□ □□□□□□□□□□.

Study points

- List core OS functions.
- Relate OS abstraction to embedded application development.
- Mention RTOS-specific timing and task services.

Engineering / workplace application: An RTOS supports predictable control tasks in automotive, industrial, and instrumentation products.

Illustrative example: A task can request a delay from the RTOS while another ready task uses the processor.

Common mistake: A general-purpose OS response average may be acceptable for a desktop but insufficient for a hard timing requirement.

M4.02 — Types and features of Operating Systems (1 hours)

Concept and explanation

Operating systems can be classified as batch, time-sharing, distributed, network, embedded, mobile, general-purpose, or real-time systems. Features differ in scheduling, resource sharing, user interaction, reliability, determinism, and footprint. A real-time system is defined by meeting timing constraints, not merely by running quickly. Hard real-time misses may be unacceptable; soft real-time misses may degrade service.

Malayalam support: OS types batch, time-sharing, distributed, network, embedded, mobile, general-purpose, real-time □□□□□□□□□□ □□□□□. Real-time system fast □□□□□□□□□□□□□□□□□□□□ timing constraint meet □□□□□□□.

Study points

- Classify OS types by interaction and timing.
- Differentiate hard and soft real-time behaviour.
- Relate footprint and determinism to embedded use.

Engineering / workplace application: An industrial protection action may need hard timing, while a multimedia display may tolerate occasional soft timing variation.

Illustrative example: A motor over-current shutdown task may have a strict deadline, while a diagnostic log task may be lower priority.

Common mistake: Calling any fast operating system real-time ignores deadlines and worst-case response.

M4.03 — Tasks, processes and threads (3 hours)

Concept and explanation

A process is an executing program with its own resources; a thread is an execution path within a process and may share process resources. A task in an RTOS is a schedulable unit with state, priority, stack, and timing behaviour. States commonly include ready, running, blocked, suspended, and terminated. Context switching saves one execution context and restores another. Shared resources require synchronisation.

Malayalam support: Process □□□□□□□ resources □□□□ program □□□; thread process-□□□□ execution path □□□; RTOS task schedulable unit □□□. Ready, running, blocked, suspended □□□□□ states □□□□□□□□.

Study points

- Compare process, thread, and task.
- Describe task states and context switching.
- Identify when shared data needs protection.

Engineering / workplace application: Separate tasks can handle sensing, control, communication, and diagnostics in an embedded product.

Illustrative example: A sensor task may block waiting for a timer, allowing a ready control task to run.

Common mistake: Creating many tasks without stack, priority, and timing analysis can exhaust memory and reduce determinism.

M4.04 — Multiprocessing and multitasking (1 hours)

Concept and explanation

Multiprocessing uses more than one processing element or core, while multitasking interleaves or schedules multiple tasks on one or more processors. Concurrency introduces shared-data, ordering, timing, and resource-ownership issues. An RTOS scheduler must preserve priority and response requirements while managing context switches and interrupts.

Malayalam support: Multiprocessing multiple processing elements □□□□□□□□□□□□; multitasking □□ tasks schedule □□□□□□□. Concurrency-□ shared data, ordering, timing, resource ownership □□□□□□□□□□ □□□□□□□□□.

Study points

- Differentiate multiprocessing and multitasking.
- State one benefit and one concurrency risk.
- Relate scheduler and processor availability.

Engineering / workplace application: Multi-core embedded controllers can separate control and communication workloads while a scheduler coordinates tasks.

Illustrative example: A communication task and control task may run concurrently but must protect a shared buffer.

Common mistake: Parallel execution does not remove the need for synchronisation or timing analysis.

M4.05 — Task-scheduling algorithms (3 hours)

Concept and explanation

Task scheduling decides which ready task executes. Priority-based preemptive scheduling can run a higher-priority task when it becomes ready; round-robin shares time among equal-priority tasks; rate-monotonic reasoning assigns higher priority to tasks with shorter periods under stated assumptions. Scheduling analysis considers execution time, period, deadline, interrupt latency, blocking, and context-switch overhead.

Malayalam support: Scheduler ready tasks-□ □□□ run □□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□. Priority, preemption, round-robin, period, deadline, blocking, context switch overhead □□□□□□ □□□□□□□□□□□.

Study points

- Explain priority and round-robin scheduling.
- Relate period, deadline, and execution time.
- Mention priority inversion and blocking as concerns.

Engineering / workplace application: Control loops, communication servicing, and safety tasks receive priorities based on deadlines and consequences.

Illustrative example: A periodic 1 ms safety task generally needs a response analysis different from a 100 ms display-refresh task.

Common mistake: Assigning priority only by perceived importance without timing analysis can miss deadlines.

M4.06 — Task communication and synchronization (4 hours)

Concept and explanation

Tasks communicate through shared memory, message queues, mailboxes, pipes, event flags, or signals. Synchronisation controls ordering and safe access using mutexes, semaphores, critical sections, and event mechanisms. A mutex protects ownership of a shared resource; a semaphore can represent a count or event; a message queue transfers data. Deadlock, starvation, race conditions, and priority inversion must be prevented or bounded.

Malayalam support: Tasks message queue, shared memory, semaphore, mutex, event flags □□□□□□ □□□ communicate □□□□□□□. Race condition, deadlock, starvation, priority inversion □□□□□□□□□□.

Study points

- Compare mutex, semaphore, and message queue.
- Define race condition and deadlock.
- Use ownership and bounded critical sections.

Engineering / workplace application: Sensor, control, and communication tasks exchange measurements and commands through RTOS primitives.

Illustrative example: A producer task places sensor data in a queue and a consumer task receives it without directly sharing an unprotected buffer.

Common mistake: Holding a mutex while waiting for an unrelated event can create deadlock or long blocking.

M4.07 — Device drivers (1 hours)

Concept and explanation

A device driver provides an interface between the operating system or application and a hardware peripheral. It configures registers, manages interrupts or DMA where applicable, exposes read/write/control operations, handles errors, and hides device-specific details behind a defined interface. A driver must respect timing, concurrency, and resource ownership.

Malayalam support: Device driver OS/application-□□□ hardware peripheral-□□□ □□□□□□ interface □□□. Registers, interrupts, read/write/control, errors, timing, concurrency □□□□□□ manage □□□□□□□.

Study points

- State the role of a driver and its interface.
- Relate driver operation to registers and interrupts.
- Include error and concurrency handling.

Engineering / workplace application: Drivers support serial ports, displays, sensors, storage, network interfaces, and motor-control peripherals.

Illustrative example: A serial driver can buffer received bytes in an ISR and expose a safe read operation to application tasks.

Common mistake: Directly changing hardware registers from many tasks without a driver policy creates conflicts.

M4.08 — Functional and nonfunctional requirements when selecting an RTOS (1 hours)

Concept and explanation

Functional requirements include task management, scheduling, timers, inter-task communication, synchronisation, device-driver support, interrupt handling, and memory management. Nonfunctional requirements include determinism, worst-case latency, footprint, reliability, safety certification, portability, tooling, debugging, community or vendor support, licensing, power, and cost. Selection should match the application risk and deadline, then be verified with a representative workload.

Malayalam support: RTOS selection functional features □□□□□□ □□□□□□□□□. Determinism, latency, footprint, reliability, safety certification, tools, support, licensing, power, cost □□□□□□ nonfunctional requirements □□□.

Study points

- Separate functional and nonfunctional requirements.
- Relate RTOS choice to deadlines and safety risk.
- Verify with representative workload and evidence.

Engineering / workplace application: Automotive, medical, and industrial products may require certification evidence and predictable worst-case behaviour.

Illustrative example: A selection matrix can score candidate RTOS options for scheduling, latency, memory footprint, tooling, certification, and vendor support.

Common mistake: Choosing an RTOS only because it is popular or free may ignore timing, safety, and support requirements.

Module summary: RTOS concepts are about predictable timing and safe coordination of tasks, resources, interrupts, and device interfaces.

Formula and Notation Bank

Formula note: The supplied syllabus is primarily procedural or conceptual and does not prescribe a compulsory numerical formula bank. Focus on the official terminology, sequences, records, and practical demonstrations listed in the modules.

Diagram Library for Rapid Revision

Module 2: AVR

Open the module to view its labelled learning-map diagram.

Module 3: Microcontroller

Open the module to view its labelled learning-map diagram.

Module 4: Real-Time

Open the module to view its labelled learning-map diagram.

Open the module to view its labelled learning-map diagram.

Official Activities and Practical Requirements

Official activities / self-learning

- The supplied syllabus does not provide a separate official self-learning activity list for this theory course. Use the topic procedures, worked examples, and module questions in this lesson for structured study.

Practical requirements / equipment

- The supplied syllabus does not prescribe separate practical equipment for this theory course.

Assessment Methodology

Official assessment information is reproduced below from the supplied syllabus.

- Source anomaly: The supplied Course Outcomes table lists Series Test as 2 hours, while the course outline separately lists Series Test I and Series Test II as 1 hour within Modules 2

and 4; both official entries are preserved.
- No separate CIE/ESE mark distribution is stated in the supplied PDF.

Module-wise Questions and Answers

module-1 — Embedded-System Concepts, Building Blocks and AVR Architecture

1. Explain Features of embedded systems. [3 marks]

An embedded system is a computer-based system designed to perform a dedicated function within a larger product. Compared with a general-purpose computer, it commonly has a defined application, constrained memory and power, real-time or predictable response needs, specialised I/O, and long service life. Systems can be classified by application, complexity, performance, connectivity, timing, and scale. Applications include appliances, vehicles, industrial controllers, medical devices, consumer products, and communication equipment.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

2. Explain Building blocks of embedded systems. [3 marks]

Building blocks include the processing core, memory, sensors, actuators, I/O subsystem, communication interface, embedded firmware, power supply, clock and reset, and protection circuits. Memory may be classified as program memory, data memory, volatile, non-volatile, internal, or external. Memory selection depends on capacity, speed, endurance, power, cost, and retention. Sensors measure physical variables; actuators produce physical action; interfaces connect the controller to the environment and other systems.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

3. Explain AVR microcontroller architecture. [3 marks]

The AVR architecture is examined through selection factors, a simplified microcontroller view, and ATmega32 resources. Important elements include registers, data memory, status register, program counter, program ROM space, I/O ports, and registers associated with I/O ports. The program counter points to the next instruction; status flags record arithmetic or logical

results; I/O direction and data registers configure and access pins. Architecture knowledge allows a C program to be related to hardware behaviour.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-2 — AVR Programming in C, Timers and Interrupts

4. Explain C data types for the AVR microcontroller. [3 marks]

AVR C data types should be selected according to range, signedness, memory use, and required operation. Integer widths, character data, Boolean conditions, and fixed-width types help make embedded code predictable. The programmer must consider overflow, truncation, volatile hardware registers, and the difference between a value and a memory-mapped register.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

5. Explain C programs for time delay and I/O operations. [3 marks]

I/O programming configures a port direction, writes output data, reads input data, and applies timing between changes. A delay can be created by calibrated loops or, more reliably, by a timer. Embedded code must account for clock frequency, instruction cycles, switch bounce, and hardware active-high or active-low logic. A good program uses named masks and avoids changing unrelated bits in a shared port register.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

6. Explain C programs for logic and arithmetic operations. [3 marks]

Embedded programs use arithmetic and logical operators to transform sensor values, generate control decisions, and manipulate bits. Bitwise AND masks inputs, OR sets selected bits, XOR toggles bits, and shifts move fields. Arithmetic should be checked for overflow, signedness, and scaling. Conditions should be written so that the hardware-safe state is explicit when a sensor value is invalid or out of range.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

7. Explain Data conversion and data serialisation. [3 marks]

Data conversion changes representation, such as ADC counts to voltage, integer to character digits, binary fields to hexadecimal, or sensor units to engineering units. Serialisation packs structured data into a byte sequence for transmission or storage; deserialisation reconstructs it. The design must define byte order, field width, sign, scaling, checksum or error detection, and framing.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-3 — Microcontroller Interfacing with External Peripherals

8. Explain RS232, serial port, LCD and keyboard interfacing. [3 marks]

RS232 interfacing requires suitable voltage-level conversion because microcontroller logic levels and RS232 signal levels differ. Serial programming configures baud rate, frame format, transmit/receive control, and status flags. LCD interfacing uses command and data sequences, enable timing, and data/control lines. Keyboard interfacing scans rows and columns or reads key signals and must handle debounce.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

9. Explain ADC, DAC and sensor interfacing. [3 marks]

An ADC converts an analogue sensor voltage into a digital code for the microcontroller. Resolution, reference voltage, sampling time, input range, quantisation, and noise affect the result. A DAC converts a digital value to an analogue output for a control or signal-generation stage. Sensor interfacing also requires calibration, filtering, signal conditioning, correct grounding, and safe voltage levels. C code selects a channel, starts conversion, waits for completion, reads the result, scales it, and applies a control decision.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

module-4 — Real-Time Operating System Concepts

10. Explain Functions of an Operating System. [3 marks]

An operating system manages processor time, memory, files or resources, input/output devices, protection, user or task interfaces, and error handling. In an embedded or real-time system, the OS also provides task management, timing services, inter-task communication, synchronisation, and device access. The OS creates an abstraction so applications can use resources through defined interfaces rather than controlling every hardware detail directly.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

11. Explain Types and features of Operating Systems. [3 marks]

Operating systems can be classified as batch, time-sharing, distributed, network, embedded, mobile, general-purpose, or real-time systems. Features differ in scheduling, resource sharing, user interaction, reliability, determinism, and footprint. A real-time system is defined by meeting timing constraints, not merely by running quickly. Hard real-time misses may be unacceptable; soft real-time misses may degrade service.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

12. Explain Tasks, processes and threads. [3 marks]

A process is an executing program with its own resources; a thread is an execution path within a process and may share process resources. A task in an RTOS is a schedulable unit with state, priority, stack, and timing behaviour. States commonly include ready, running, blocked, suspended, and terminated. Context switching saves one execution context and restores another. Shared resources require synchronisation.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

13. Explain Multiprocessing and multitasking. [3 marks]

Multiprocessing uses more than one processing element or core, while multitasking interleaves or schedules multiple tasks on one or more processors. Concurrency introduces

shared-data, ordering, timing, and resource-ownership issues. An RTOS scheduler must preserve priority and response requirements while managing context switches and interrupts.

Exam focus: State the definition or purpose, explain the working or sequence, and add one application or caution.

Practice Model Question Paper

Practice Model Paper — Not an Official Examination Paper

This paper is generated from the supplied module coverage for revision and does not claim to reproduce the official examination pattern.

1. **1.** Define or explain *Features of embedded systems* with one relevant application. (5 marks)
2. **2.** Define or explain *Building blocks of embedded systems* with one relevant application. (5 marks)
3. **3.** Define or explain *C data types for the AVR microcontroller* with one relevant application. (5 marks)
4. **4.** Define or explain *C programs for time delay and I/O operations* with one relevant application. (5 marks)
5. **5.** Define or explain *RS232, serial port, LCD and keyboard interfacing* with one relevant application. (5 marks)
6. **6.** Define or explain *ADC, DAC and sensor interfacing* with one relevant application. (5 marks)
7. **7.** Define or explain *Functions of an Operating System* with one relevant application. (5 marks)
8. **8.** Define or explain *Types and features of Operating Systems* with one relevant application. (5 marks)

> **Writing advice:** Use headings, standard terminology, and labelled diagrams or procedures where relevant.

Quick Revision

Module-wise key points

- **Embedded-System Concepts, Building Blocks and AVR Architecture:** Embedded systems are understood by connecting application constraints to hardware blocks, firmware, memory, I/O, and architecture.
- **AVR Programming in C, Timers and Interrupts:** AVR C programming becomes reliable when data types, bit operations, timing, registers, flags, and ISR boundaries are explicitly controlled.
- **Microcontroller Interfacing with External Peripherals:** Peripheral interfacing is a complete path from electrical signal and protocol to register configuration, timing, C code, and validated output.
- **Real-Time Operating System Concepts:** RTOS concepts are about predictable timing and safe coordination of tasks, resources, interrupts, and device interfaces.

Exam checklist

- Write the official terminology before adding explanation.
- Use a labelled diagram or step sequence when it improves the answer.
- Check conditions, boundaries, safety, hygiene, and documentation points.
- Separate official syllabus information from your own example.

> **Last-minute revision:** Review the coverage table, formula bank, diagram library, and common-mistake notes inside every topic.

Glossary

Technical glossary

Term	Short meaning
Features of embedded systems	An embedded system is a computer-based system designed to perform a dedicated function within a larger product. Compared with a general-purpose computer, it commonly has a defined application, constrained memory and power, real-time or predictable response nee
Building blocks of embedded systems	Building blocks include the processing core, memory, sensors, actuators, I/O subsystem, communication interface, embedded firmware, power supply, clock and reset, and protection circuits. Memory may be classified as program memory, data memory, volatile, non-v

Term	Short meaning
AVR microcontroller architecture	The AVR architecture is examined through selection factors, a simplified microcontroller view, and ATmega32 resources. Important elements include registers, data memory, status register, program counter, program ROM space, I/O ports, and registers associated w
C data types for the AVR microcontroller	AVR C data types should be selected according to range, signedness, memory use, and required operation. Integer widths, character data, Boolean conditions, and fixed-width types help make embedded code predictable. The programmer must consider overflow, trunca
C programs for time delay and I/O operations	I/O programming configures a port direction, writes output data, reads input data, and applies timing between changes. A delay can be created by calibrated loops or, more reliably, by a timer. Embedded code must account for clock frequency, instruction cycles,
C programs for logic and arithmetic operations	Embedded programs use arithmetic and logical operators to transform sensor values, generate control decisions, and manipulate bits. Bitwise AND masks inputs, OR sets selected bits, XOR toggles bits, and shifts move fields. Arithmetic should be checked for over
Data conversion and data serialisation	Data conversion changes representation, such as ADC counts to voltage, integer to character digits, binary fields to hexadecimal, or sensor units to engineering units. Serialisation packs structured data into a byte sequence for transmission or storage; deseri
Normal and CTC modes of timers	In Normal timer mode, the counter increments through its range and overflows, which can trigger a flag or interrupt. In Clear Timer on Compare (CTC) mode, the counter resets when it matches a programmed compare value, producing a repeatable timing interval. Ti
C programs for time delays and event counting using Timer/Counter	A timer can generate a delay by counting clock ticks; a counter can count external events. A C program configures the mode and registers, starts the timer, waits for a flag or services an interrupt, then clears or records the event. Time calculations must use
Interrupts in AVR	An interrupt temporarily transfers control from the main program to an Interrupt Service Routine (ISR) when an enabled event occurs. AVR interrupt sources can include external pins, timers, serial communication, ADC completion, and other peripherals. The contr
Interrupt programming in C	Interrupt programming enables the source, defines the vector or handler, writes a short ISR, clears the event condition, and safely communicates results to the main program. Shared variables may need volatile qualification and atomic access or temporary interr
RS232, serial port, LCD and keyboard interfacing	RS232 interfacing requires suitable voltage-level conversion because microcontroller logic levels and RS232 signal levels differ. Serial programming configures baud rate, frame format, transmit/receive control, and status flags. LCD interfacing uses command an
ADC, DAC and sensor interfacing	An ADC converts an analogue sensor voltage into a digital code for the microcontroller. Resolution, reference voltage, sampling time, input range, quantisation, and noise affect the result. A DAC converts a digital value to an analogue output for a control or

Term	Short meaning
Functions of an Operating System	An operating system manages processor time, memory, files or resources, input/output devices, protection, user or task interfaces, and error handling. In an embedded or real-time system, the OS also provides task management, timing services, inter-task communi
Types and features of Operating Systems	Operating systems can be classified as batch, time-sharing, distributed, network, embedded, mobile, general-purpose, or real-time systems. Features differ in scheduling, resource sharing, user interaction, reliability, determinism, and footprint. A real-time s
Tasks, processes and threads	A process is an executing program with its own resources; a thread is an execution path within a process and may share process resources. A task in an RTOS is a schedulable unit with state, priority, stack, and timing behaviour. States commonly include ready,
Multiprocessing and multitasking	Multiprocessing uses more than one processing element or core, while multitasking interleaves or schedules multiple tasks on one or more processors. Concurrency introduces shared-data, ordering, timing, and resource-ownership issues. An RTOS scheduler must pre
Task-scheduling algorithms	Task scheduling decides which ready task executes. Priority-based preemptive scheduling can run a higher-priority task when it becomes ready; round-robin shares time among equal-priority tasks; rate-monotonic reasoning assigns higher priority to tasks with sho
Task communication and synchronization	Tasks communicate through shared memory, message queues, mailboxes, pipes, event flags, or signals. Synchronisation controls ordering and safe access using mutexes, semaphores, critical sections, and event mechanisms. A mutex protects ownership of a shared res
Device drivers	A device driver provides an interface between the operating system or application and a hardware peripheral. It configures registers, manages interrupts or DMA where applicable, exposes read/write/control operations, handles errors, and hides device-specific d
Functional and nonfunctional requirements when selecting an RTOS	Functional requirements include task management, scheduling, timers, inter-task communication, synchronisation, device-driver support, interrupt handling, and memory management. Nonfunctional requirements include determinism, worst-case latency, footprint, rel

Official References and Resources

References listed in the supplied syllabus

1. Shibu K.V, Introduction to Embedded Systems, McGraw Hill, First Edition.
2. Muhammad Ali Mazidi, Sarmad Naimi, and Sepehr Naimi, The AVR Microcontroller and Embedded Systems Using Assembly and C, Pearson Education.
3. Michael J. Pont, Embedded C, Pearson Education, Second Edition.

4. Raj Kamal, Embedded Systems, McGraw Hill, Second Edition.

Official learning resources listed

- https://www.studyelectronics.in/embedded-programming-tutorial-chapter-1-beginners/
- https://www.tutorialspoint.com/embedded_systems/index.htm
- https://embeddedschool.in/avr-microcontroller-programming/

In-page Search